SDU
University of
Southern Denmark

Module 4: Convolutional neural networks and physics-informed neural networks

Tools of Artificial intelligence

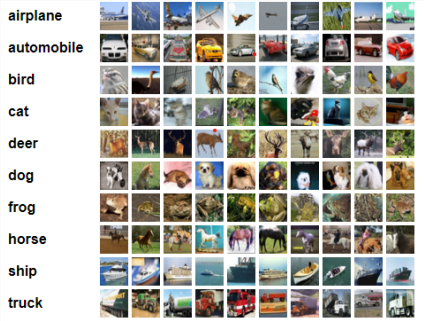
Orkun Furat

March 25 and April 8, 2026

SDU Applied AI and Data Science Unit, University of Southern Denmark, Denmark

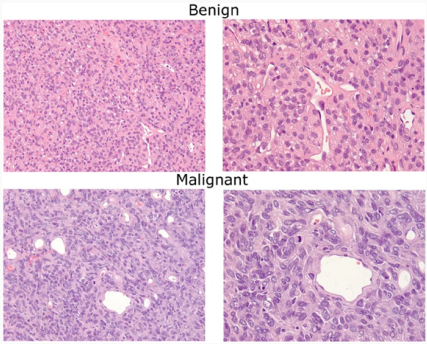
Outline

Motivation — Image classification



Natural image classification

- Dataset: CIFAR-10
- Each image belongs to one of 10 classes
- Goal: automatically assign the correct label

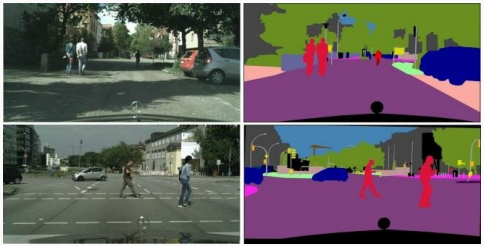


Microscopy image classification

- Example: histopathology images
- Cell patterns in tissue samples can indicate diseases
- Example task: benign vs malignant

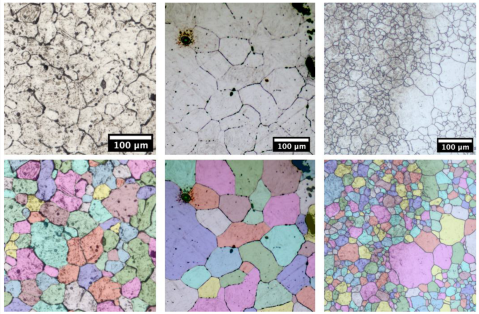
image $\Longrightarrow$ class label

Motivation — Image segmentation



Natural image segmentation

- Example dataset: Cityscapes
- Each pixel is assigned a class
- Goal: identify objects and regions in the scene



Microscopy image segmentation

- Example: microstructure images
- Detect and separate individual crystals (grains)
- Enables quantitative microstructure analysis

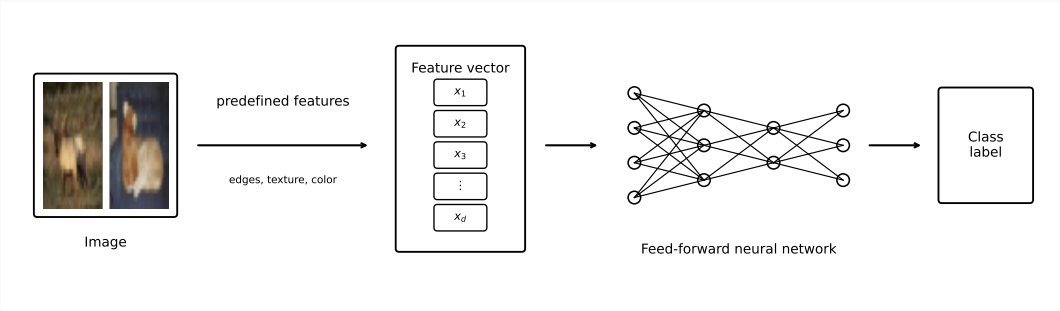
image $\Longrightarrow$ labeled image (each pixel is classified)

Learning objectives

After this module, you should be able to:

- explain the difference between **image classification** and **image segmentation**
- understand how **convolution filters** extract features from images
- describe the basic structure of artificial neural networks with dense layers
- describe the basic structure of common **convolutional neural networks (CNN)**
- use CNNs for
  - image classification
  - image segmentation
- understand how physical equations can be integrated into neural networks' training process

Image classification — Overview

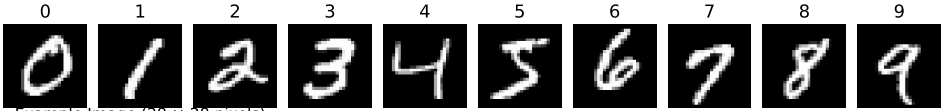


Conventional image classification pipelines:

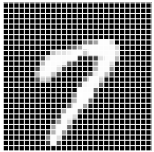
- extract (manually designed) features from images
- each image is represented by a feature vector
- classifiers are trained that make their decision based on the feature vector (see Module 2)

Image classification — Feature extraction

MNIST Dataset: Handwritten Digit Images



Example Image (28 × 28 pixels)



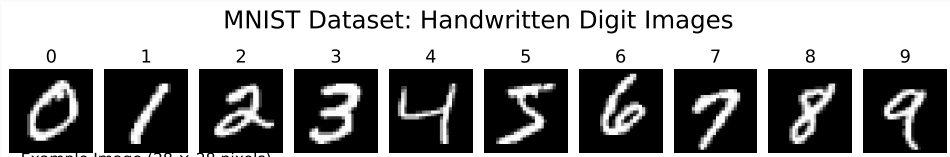
Dataset properties:

- Total images: 70,000
- Training images: 60,000
- Test images: 10,000

- Image size: 28 × 28 pixels
- Pixel values: grayscale (0–255)
- 10 classes: digits 0–9

Each image can be interpreted as a 784-dimensional feature vector.

Image classification — Feature extraction



Dataset properties:

- Total images: 70,000
- Training images: 60,000
- Test images: 10,000

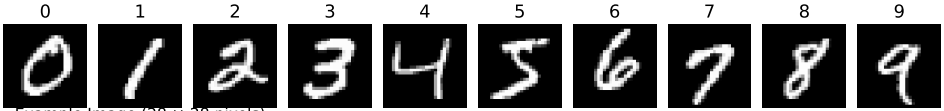
- Image size: 28 × 28 pixels
- Pixel values: grayscale (0–255)
- 10 classes: digits 0–9

Each image can be interpreted as a 784-dimensional feature vector.

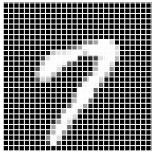
- We normalize images by dividing pixel values 0,...,255 by 255 $\Longrightarrow$ values in the interval [0,1]

Image classification — Feature extraction

MNIST Dataset: Handwritten Digit Images



Example Image (28 × 28 pixels)



Dataset properties:

- Total images: 70,000
- Training images: 60,000
- Test images: 10,000

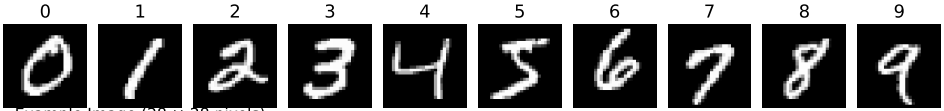
- Image size: 28 × 28 pixels
- Pixel values: grayscale (0–255)
- 10 classes: digits 0–9

Each image can be interpreted as a 784-dimensional feature vector.

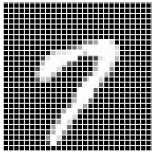
- We normalize images by dividing pixel values 0,...,255 by 255 $\Longrightarrow$ values in the interval [0,1]
- Using the full image as a feature vector leads to high dimensionality and redundancy, motivating feature extraction

Image classification — Feature extraction

MNIST Dataset: Handwritten Digit Images



Example Image (28 × 28 pixels)



Dataset properties:

- Total images: 70,000
- Training images: 60,000
- Test images: 10,000

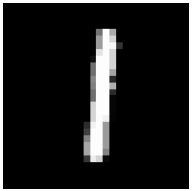
- Image size: 28 × 28 pixels
- Pixel values: grayscale (0–255)
- 10 classes: digits 0–9

Each image can be interpreted as a 784-dimensional feature vector.

- We normalize images by dividing pixel values 0,...,255 by 255 $\Longrightarrow$ values in the interval [0,1]
- Using the full image as a feature vector leads to high dimensionality and redundancy, motivating feature extraction
- **We begin to focus on binary classification to distinguish between 1 and 7.**

Image classification — Feature extraction

Digit 1



average gray value

0.071

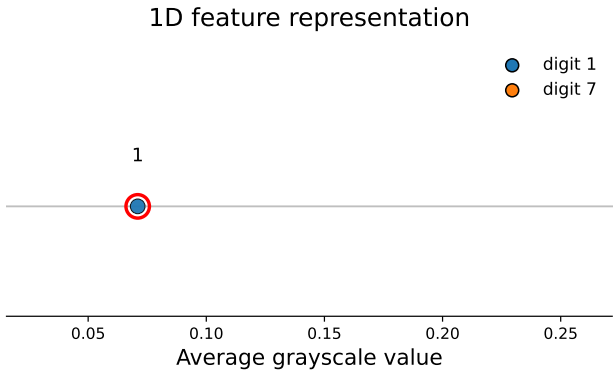


Image classification — Feature extraction

Digit 7

average gray value

0.139

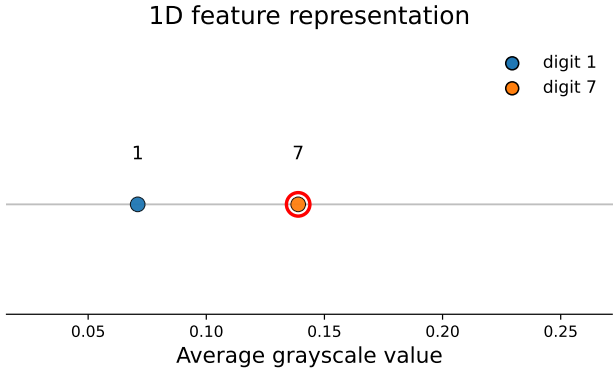
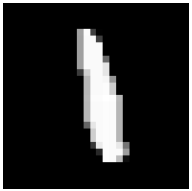


Image classification — Feature extraction

Digit 1



average gray value

0.107

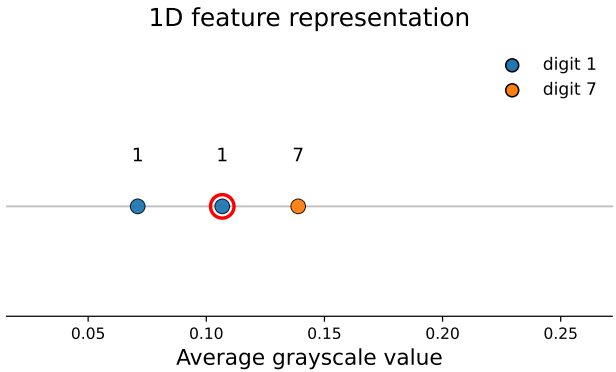


Image classification — Feature extraction

Digit 7

average gray value

0.148

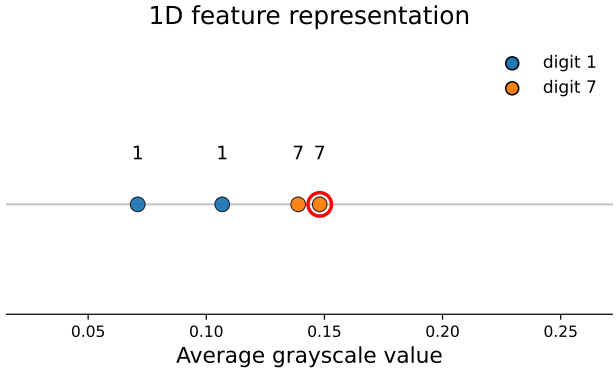
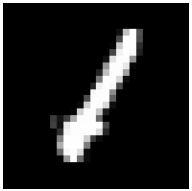


Image classification — Feature extraction

Digit 1



average gray value

0.095

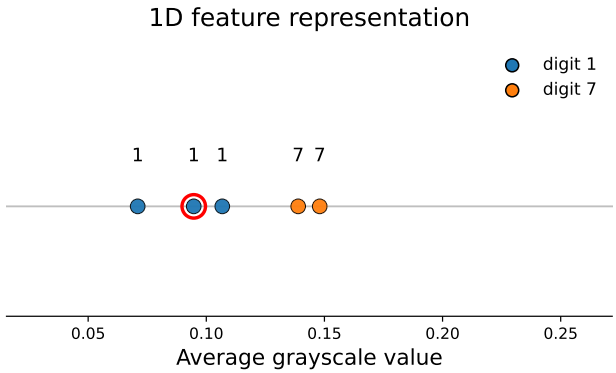


Image classification — Feature extraction

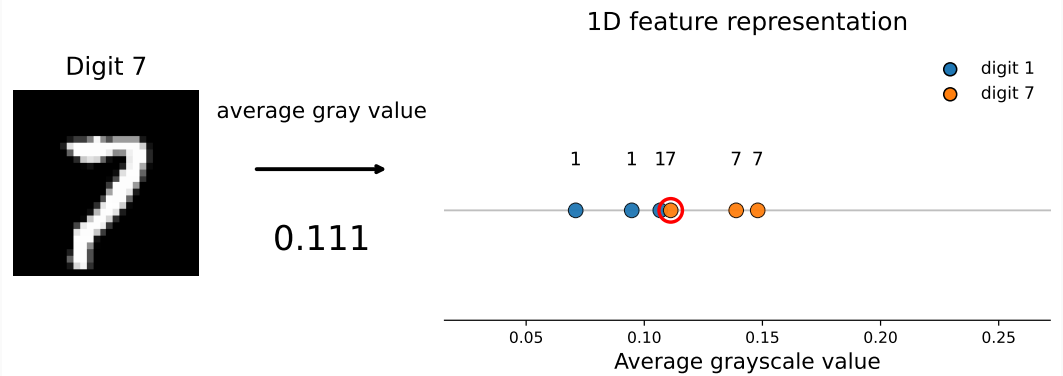
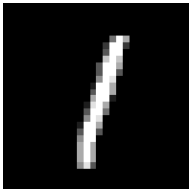


Image classification — Feature extraction

Digit 1



average gray value

0.065

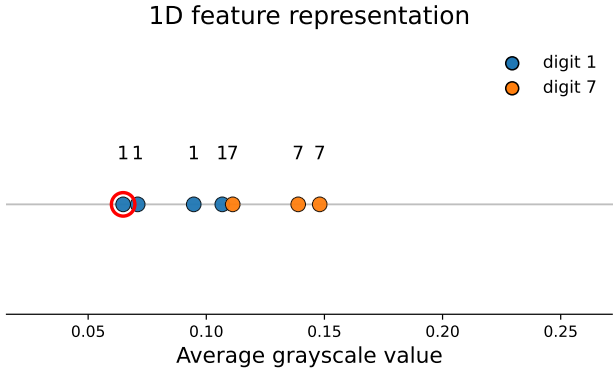
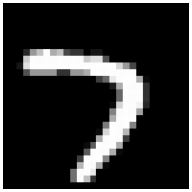


Image classification — Feature extraction

Digit 7



average gray value

0.133

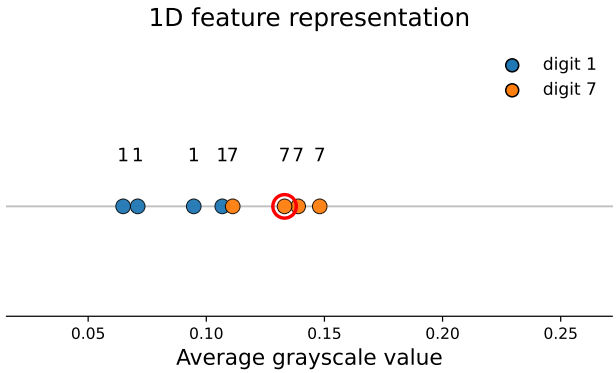


Image classification — Feature extraction

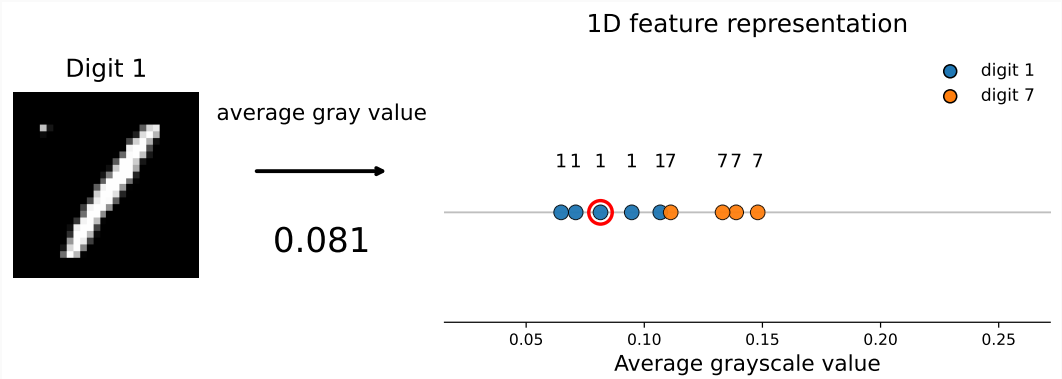


Image classification — Feature extraction

Digit 7

average gray value

0.133

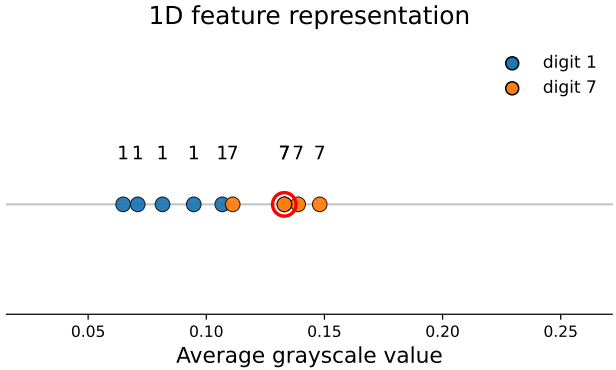


Image classification — Feature extraction

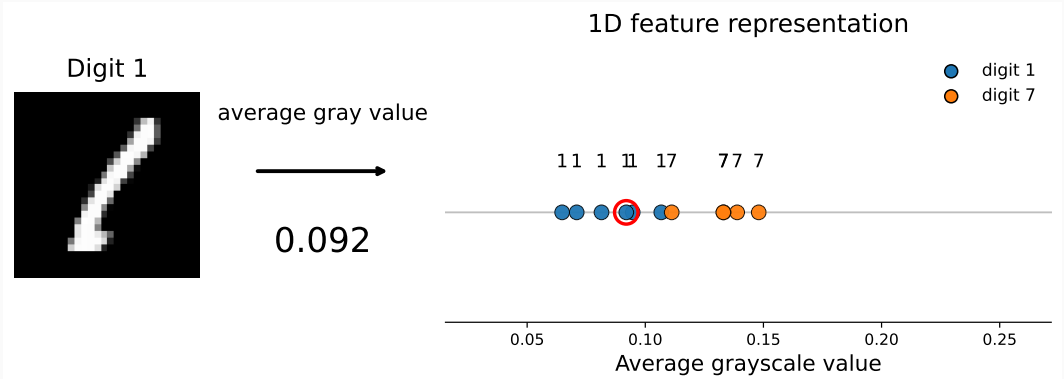
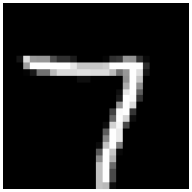


Image classification — Feature extraction

Digit 7



average gray value

0.092

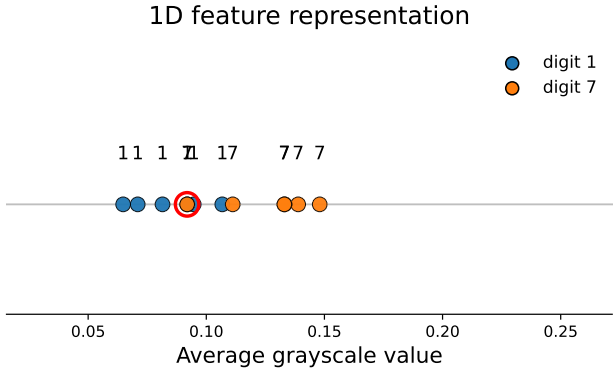


Image classification — Feature extraction

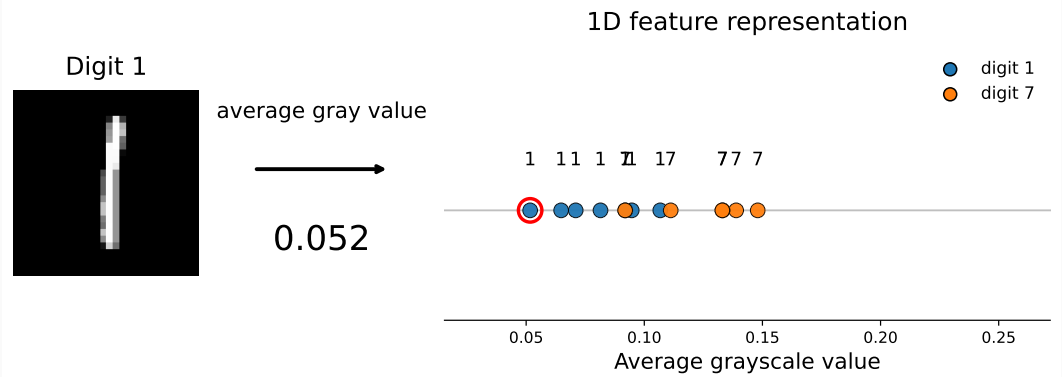
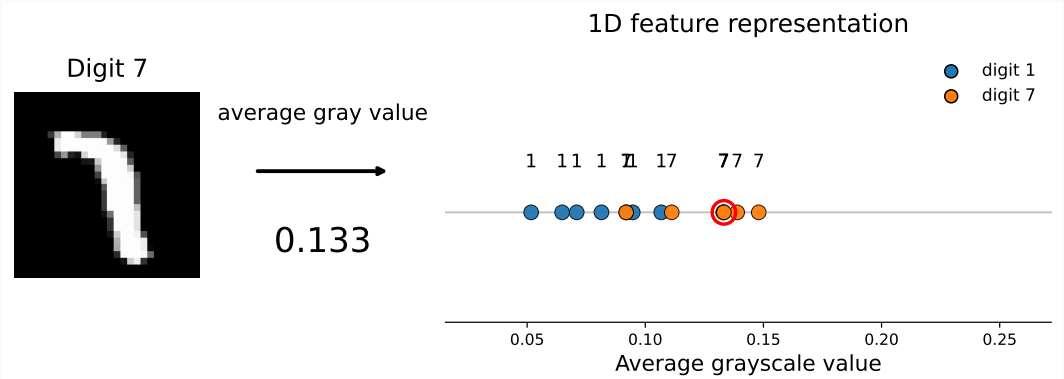
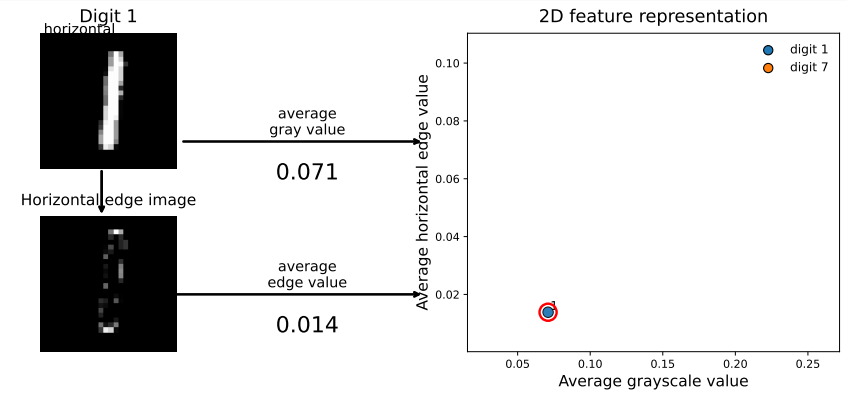


Image classification — Feature extraction



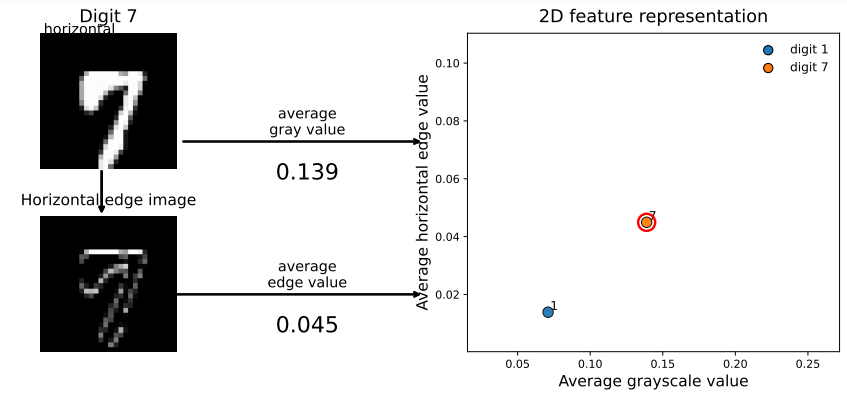
This single feature may be insufficient for distinguishing between 1 and 7.

Image classification — Feature extraction



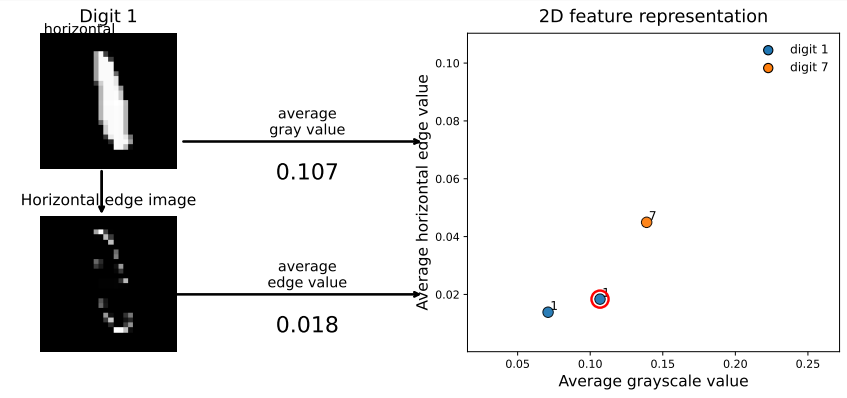
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



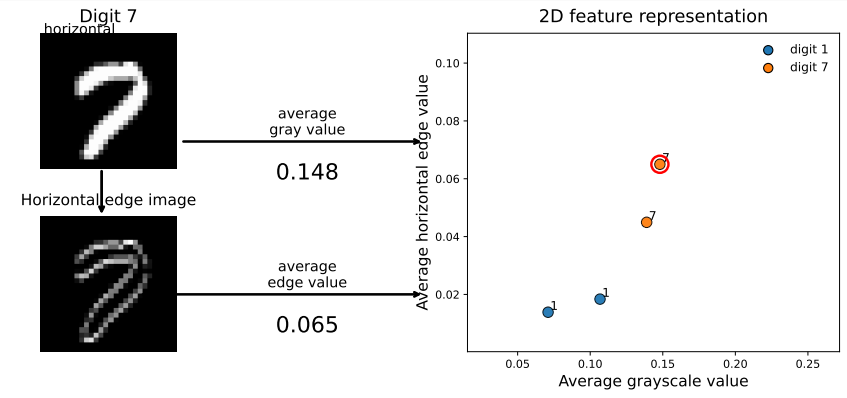
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



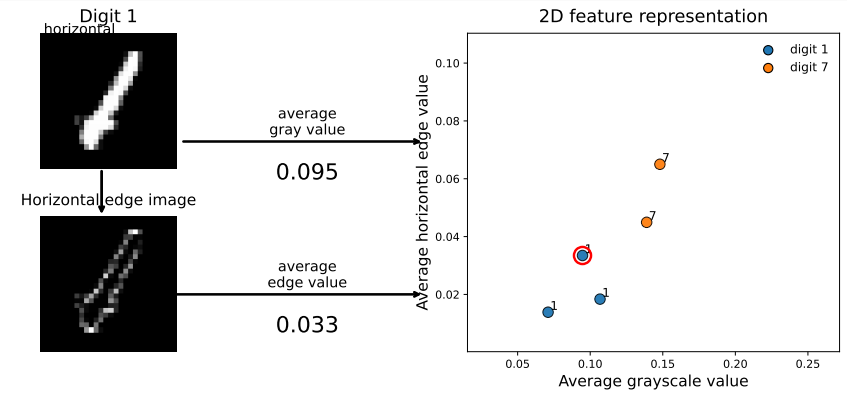
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



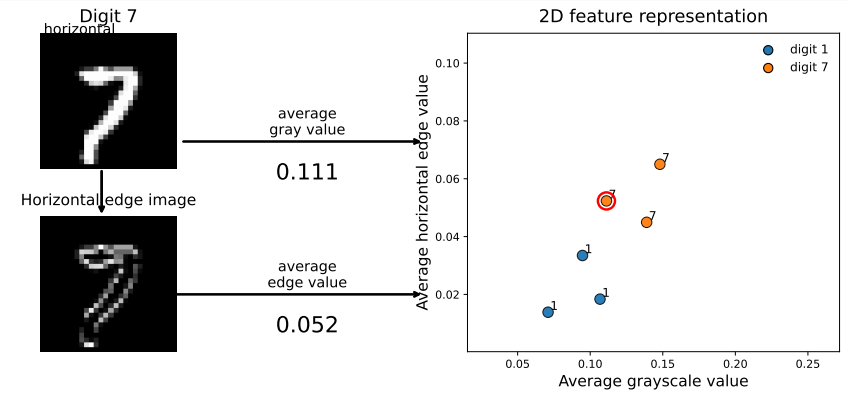
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



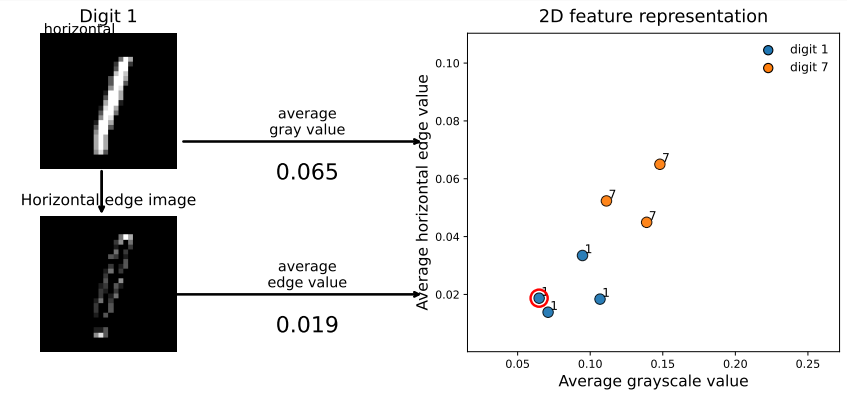
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



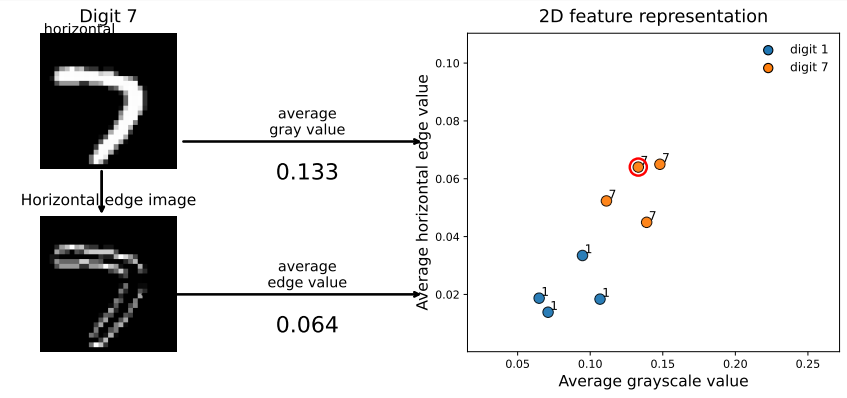
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



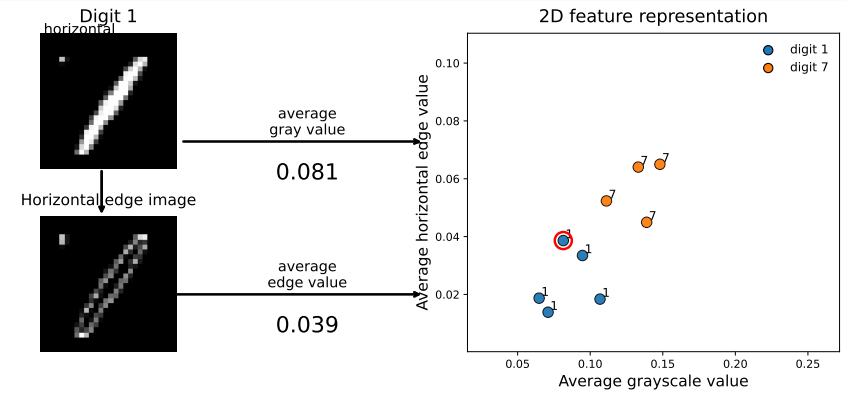
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



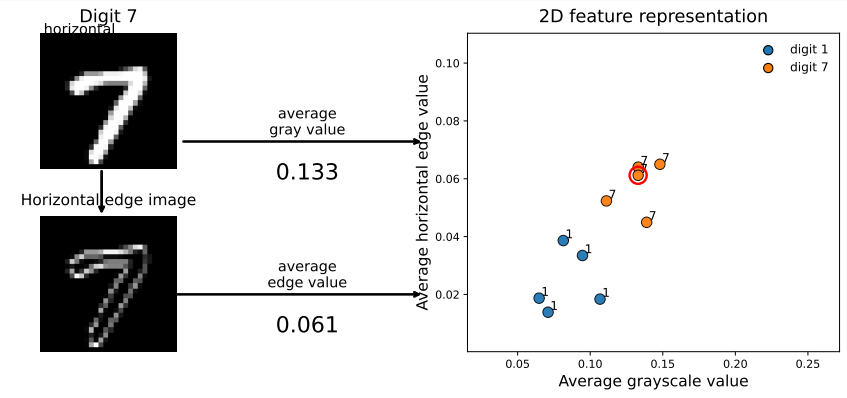
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



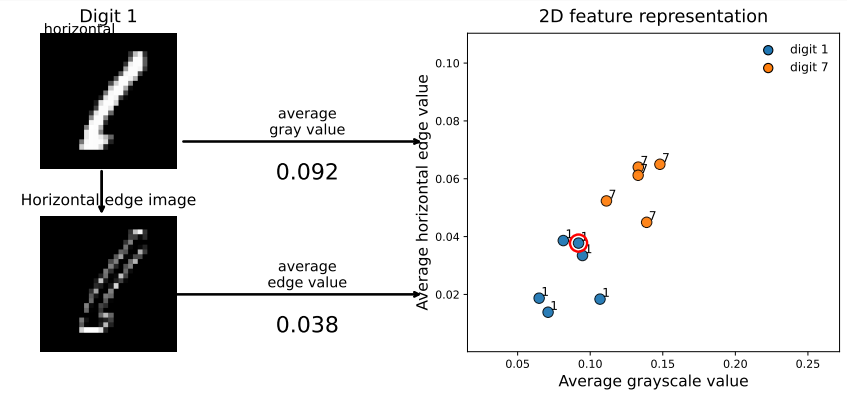
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



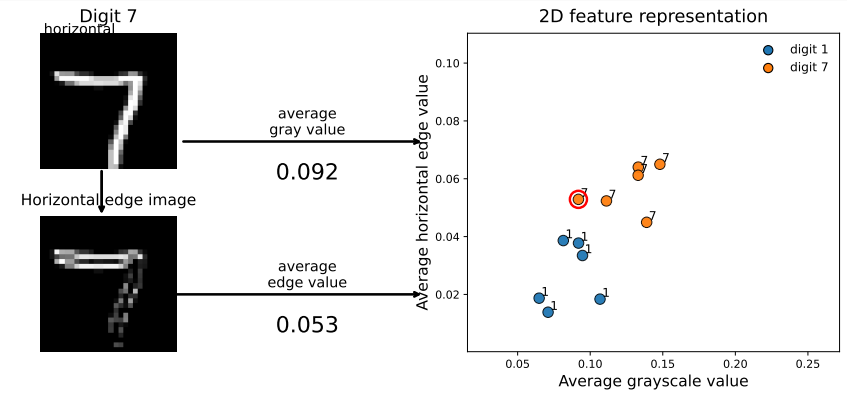
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



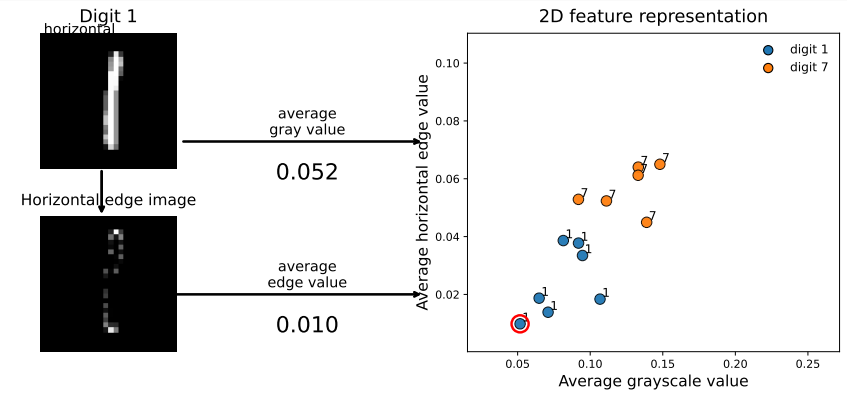
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



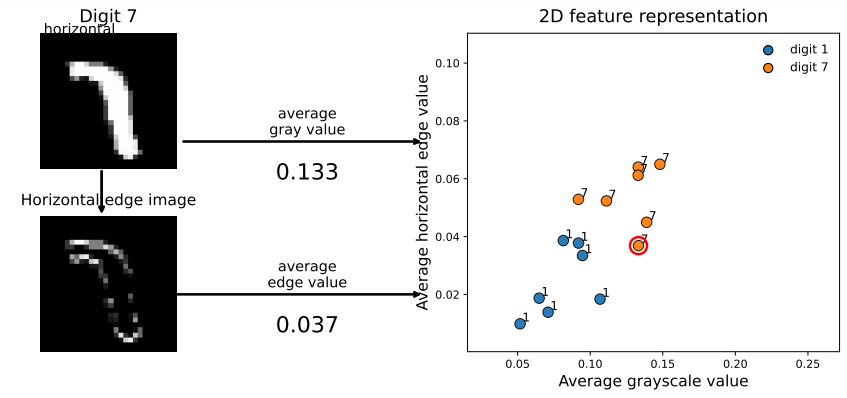
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



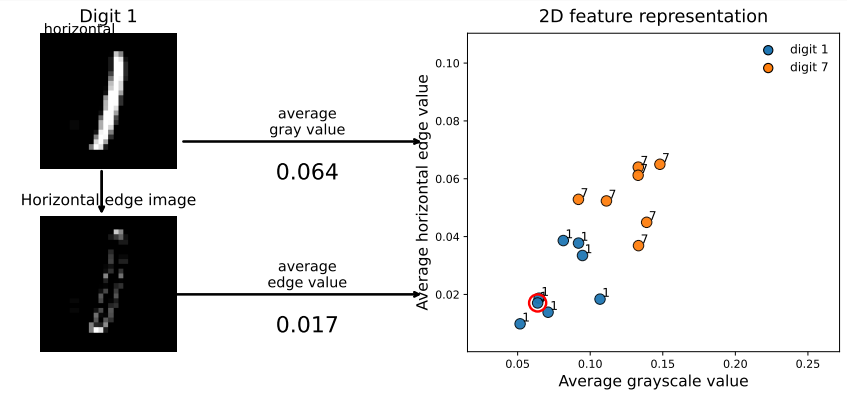
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



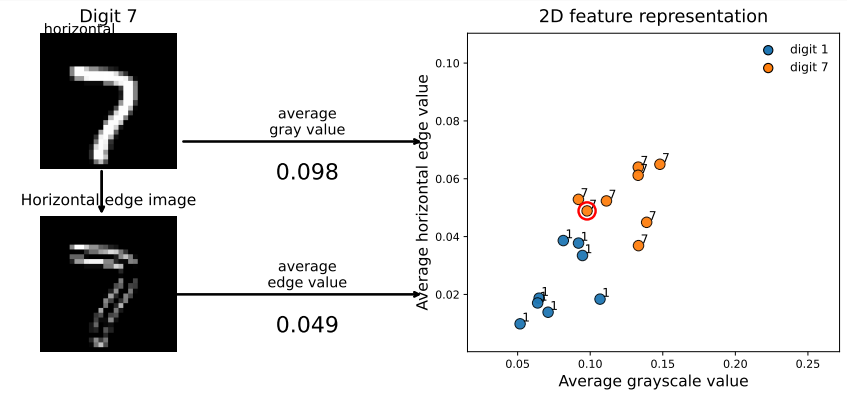
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



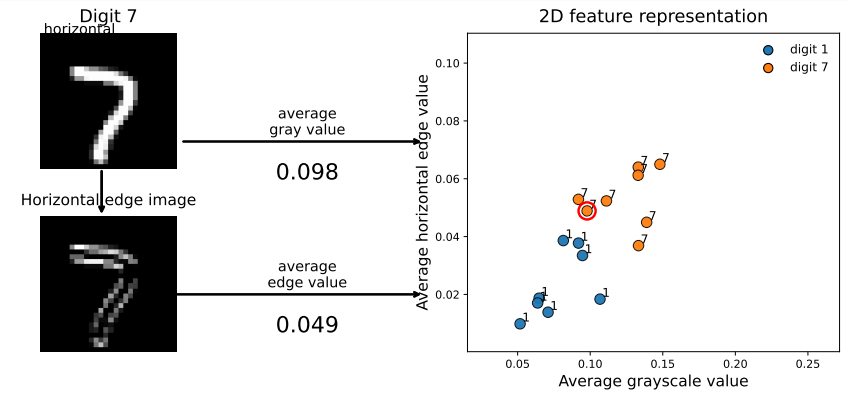
- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.

Image classification — Feature extraction



- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.
- Classifier can be trained to distinguish between both classes

Image classification — Feature extraction



- With an additional feature, resulting 2D feature vectors of 1 and 7 are spatially separated.
- Classifier can be trained to distinguish between both classes
- More details on feature extraction will follow later.

Exercise: Dot product and matrix vector multiplication

- Let $x = \begin{bmatrix} 2 \\ 5 \\ 3 \end{bmatrix}$ and $w = \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix}$. Compute the dot product $w^\top x$.
- Let $W = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$. Compute the matrix-vector product Wx.
- Let $W \in \mathbb{R}^{n \times d}$ and $x \in \mathbb{R}^d$. What is the dimension of Wx

Image classification — Artificial neural networks

- Feed-forward neural networks are built from computational units called *neurons*

Image classification — Artificial neural networks

- Feed-forward neural networks are built from computational units called *neurons*
- A dense neuron takes input values $x_1, \ldots, x_d$, forms a weighted sum, adds a bias, and applies an activation function

$$y_{\text{out}} = f\left(\sum_{\ell=1}^{d} w_\ell x_\ell + b\right) = f(w^\top x + b) \tag{1}$$

Image classification — Artificial neural networks

- Feed-forward neural networks are built from computational units called *neurons*
- A dense neuron takes input values $x_1, \ldots, x_d$, forms a weighted sum, adds a bias, and applies an activation function

$$y_{\text{out}} = f\left(\sum_{\ell=1}^{d} w_\ell x_\ell + b\right) = f(w^\top x + b) \tag{1}$$

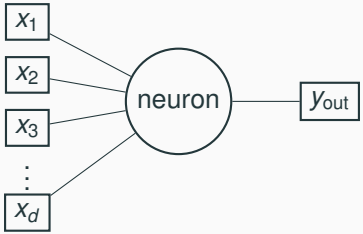
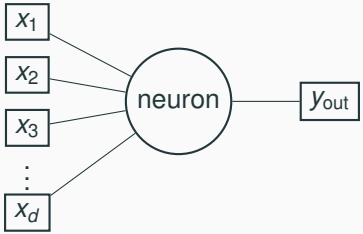


Image classification — Artificial neural networks

- Feed-forward neural networks are built from computational units called *neurons*
- A dense neuron takes input values $x_1, \ldots, x_d$, forms a weighted sum, adds a bias, and applies an activation function

$$y_{\text{out}} = f\left(\sum_{\ell=1}^{d} w_\ell x_\ell + b\right) = f(w^\top x + b) \tag{1}$$

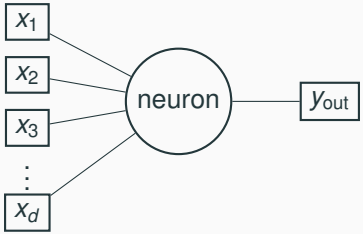


- The weight vector $w = (w_1, \ldots, w_d)^\top$ and the bias b are trainable parameters

Image classification — Artificial neural networks

- Feed-forward neural networks are built from computational units called *neurons*
- A dense neuron takes input values $x_1, \ldots, x_d$, forms a weighted sum, adds a bias, and applies an activation function

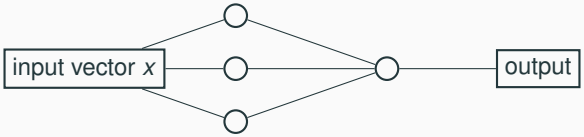
$$y_{\text{out}} = f\left(\sum_{\ell=1}^{d} w_\ell x_\ell + b\right) = f(w^\top x + b) \tag{1}$$



- The weight vector $w = (w_1, \ldots, w_d)^\top$ and the bias b are trainable parameters
- Typical activation functions are `tanh`, sigmoid, or ReLU

Image classification — Artificial neural networks

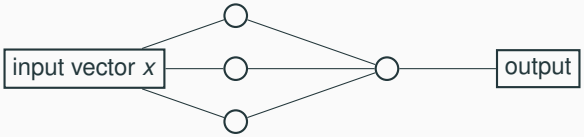
A dense layer is an aggregation of such neurons



- First layer is the input layer; last layer is the output layer. Intermediate layers are hidden layers.

Image classification — Artificial neural networks

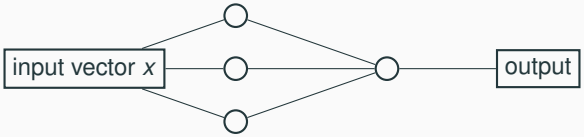
A dense layer is an aggregation of such neurons



- First layer is the input layer; last layer is the output layer. Intermediate layers are hidden layers.
- In this example, the hidden layer contains three neurons.

Image classification — Artificial neural networks

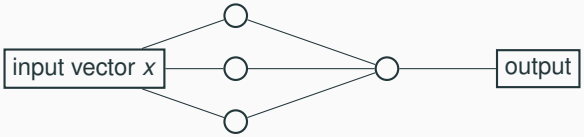
A dense layer is an aggregation of such neurons



- First layer is the input layer; last layer is the output layer. Intermediate layers are hidden layers.
- In this example, the hidden layer contains three neurons.
- Each neuron in a dense layer receives all outputs from the previous layer.

Image classification — Artificial neural networks

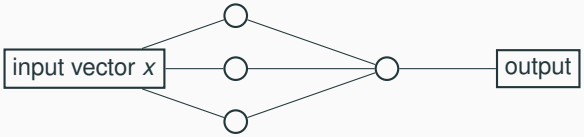
A dense layer is an aggregation of such neurons



- First layer is the input layer; last layer is the output layer. Intermediate layers are hidden layers.
- In this example, the hidden layer contains three neurons.
- Each neuron in a dense layer receives all outputs from the previous layer.
- Therefore, dense layers are also called *fully connected layers*.

Image classification — Artificial neural networks

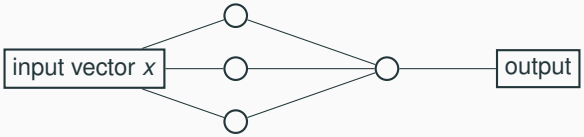
A dense layer is an aggregation of such neurons



- First layer is the input layer; last layer is the output layer. Intermediate layers are hidden layers.
- In this example, the hidden layer contains three neurons.
- Each neuron in a dense layer receives all outputs from the previous layer.
- Therefore, dense layers are also called *fully connected layers*.
- Hidden layers output feature vectors, while the final layer produces the prediction.

Image classification — Artificial neural networks

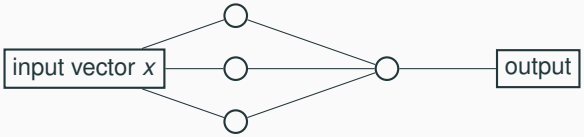
A dense layer is an aggregation of such neurons



- First layer is the input layer; last layer is the output layer. Intermediate layers are hidden layers.
- In this example, the hidden layer contains three neurons.
- Each neuron in a dense layer receives all outputs from the previous layer.
- Therefore, dense layers are also called *fully connected layers*.
- Hidden layers output feature vectors, while the final layer produces the prediction.
- In this example, the hidden layer has three neurons with weight vectors $\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3$ and biases b_1, b_2, b_3. These neurons output $y_i = f(\mathbf{w}_i^\top x + b_i)$.

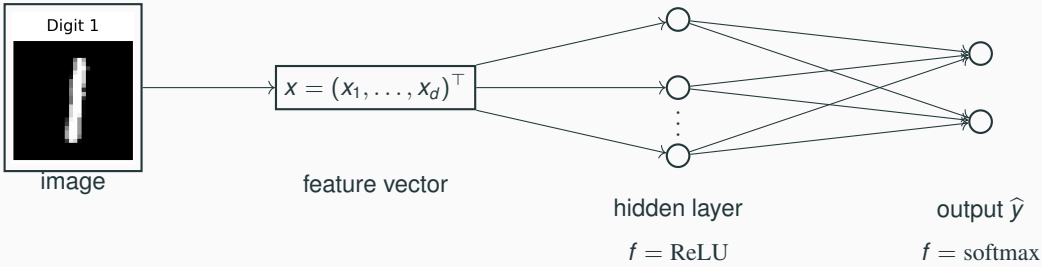
Image classification — Artificial neural networks

A dense layer is an aggregation of such neurons



- First layer is the input layer; last layer is the output layer. Intermediate layers are hidden layers.
- In this example, the hidden layer contains three neurons.
- Each neuron in a dense layer receives all outputs from the previous layer.
- Therefore, dense layers are also called *fully connected layers*.
- Hidden layers output feature vectors, while the final layer produces the prediction.
- In this example, the hidden layer has three neurons with weight vectors $\mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3$ and biases b_1, b_2, b_3. These neurons output $y_i = f(\mathbf{w}_i^\top x + b_i)$.
- Alternative, representation $y = (y_1, y_2, y_3)^\top = f(Wx + b)$ with $b = (b_1, b_2, b_3)^\top$ and W being the matrix with rows $\mathbf{w}_1^\top, \mathbf{w}_2^\top, \mathbf{w}_3^\top$.

Image classification — Activasion functions for classification



- The dense layer computes a hidden representation using the ReLU activation function. ReLU (Rectified Linear Unit) is defined as $\mathrm{ReLU}(x) = \max(0, x)$, which introduces non-linearity by outputting the input if positive, and zero otherwise. $y_{\text{hidden}} = \mathrm{ReLU}(Wx + b)$
- Within the output layer a score vector z (logits) is computed $z = W_{\text{out}} y_{\text{hidden}} + b_{\text{out}}$. Each entry is a score of associating the input x with a class
- The softmax function converts these scores into class probabilities. Softmax is defined as $\widehat{y}_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$, ensuring the outputs sum to 1 and represent probabilities.
- $\widehat{y}_i$ is the network's certainty that the feature vector x is associated with class i.

Image classification — Training

Training data

Each training sample consists of (x, y)

- x : feature vector extracted from an image
- y : class label

One-hot encoding of labels

For K classes, labels are represented as vectors $y = (y_1, \ldots, y_K)^\top$

Example (2 classes):class 1 $\rightarrow (1, 0)$ class 2 $\rightarrow (0, 1)$

Note: For binary classification, one-hot encoding can be avoided, and labels can be represented as scalars (e.g., $y \in \{0, 1\}$).

Image classification — Training

Training data

Each training sample consists of (x, y)

- x : feature vector extracted from an image
- y : class label

One-hot encoding of labels

For K classes, labels are represented as vectors $y = (y_1, \ldots, y_K)^\top$

Example (2 classes):class 1 $\rightarrow (1, 0)$ class 2 $\rightarrow (0, 1)$

Note: For binary classification, one-hot encoding can be avoided, and labels can be represented as scalars (e.g., $y \in \{0, 1\}$).

Example (3 classes):

$$\text{class 1} \rightarrow (1, 0, 0) \qquad \text{class 2} \rightarrow (0, 1, 0) \qquad \text{class 3} \rightarrow (0, 0, 1)$$

Image classification — Training

Cross-entropy loss

For the input feature vectors $x^{(1)}, \ldots, x^{(N)}$, the network produces output vectors $\widehat{y}^{(1)}, \ldots, \widehat{y}^{(N)}$ of probabilities with $\widehat{y}^{(\ell)} = (\widehat{y}_1^{(\ell)}, \ldots, \widehat{y}_K^{(\ell)})$.

The cross-entropy loss compares a prediction $\widehat{y}$ and target y: $L(y, \widehat{y}) = -\sum_{i=1}^{K} y_i \log(\widehat{y}_i)$.

Example:

If $y = (0, 1, 0)$ and $\widehat{y} = (0.2, 0.6, 0.1)$. Then:

$$L(y, \widehat{y}) = -(0 \cdot \log(0.2) + 1 \cdot \log(0.6) + 0 \cdot \log(0.1)) = -\log(0.6)$$

Image classification — Training

Cross-entropy loss

For the input feature vectors $x^{(1)}, \ldots, x^{(N)}$, the network produces output vectors $\widehat{y}^{(1)}, \ldots, \widehat{y}^{(N)}$ of probabilities with $\widehat{y}^{(\ell)} = (\widehat{y}_1^{(\ell)}, \ldots, \widehat{y}_K^{(\ell)})$.
The cross-entropy loss compares a prediction $\widehat{y}$ and target y: $L(y, \widehat{y}) = -\sum_{i=1}^{K} y_i \log(\widehat{y}_i)$.

Training

The parameters W, b of each layer are optimized to minimize the average loss over the training data.

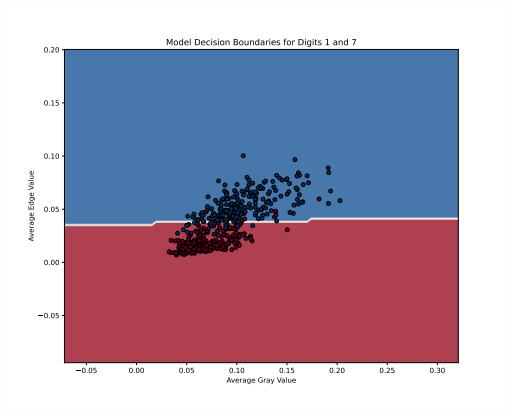
$$\text{average loss} = \frac{1}{N} \sum_{\ell=1}^{N} L(y^{(\ell)}, \widehat{y}^{(\ell)})$$

This optimization is typically performed numerically using gradient-descent-based methods (e.g., Adam).

Image classification — Evaluation

- The classifier can be evaluated using the techniques introduced in **Module 2**.
- In particular, we analyse the **decision boundary** and the **accuracy**.

Binary classification: digits 1 vs. 7



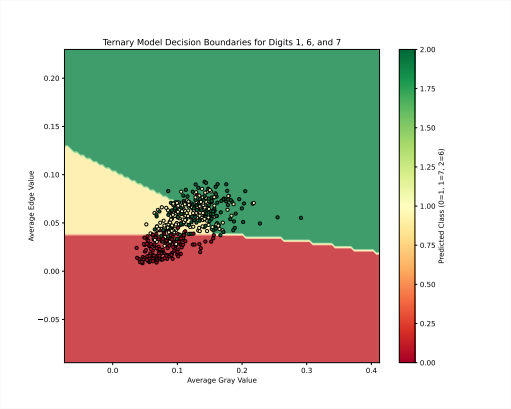
Accuracy: 0.92

- Classes are well separated
- Two features already contain useful information

Image classification — Evaluation

- The classifier can be evaluated using the techniques introduced in **Module 2**.
- In particular, we analyse the **decision boundary** and the **accuracy**.

Trinary classification: digits 1, 6 and 7



Accuracy: 0.73

- Classes overlap more strongly
- Two features are insufficient
- We will investigate how conventional feature extraction for images is performed and how it can be automated with convolutional neural networks (CNNs).

Image classification with CNNs — Convolution

- Image processing often utilizes the (discrete) convolution operation

Image classification with CNNs — Convolution

- Image processing often utilizes the (discrete) convolution operation
- For a 2D image I and a kernel k the convolution operation $*$ is defined by

$$(I * k)(x, y) = \sum_{i,j=-n}^{n} I(x - i, y - j)k(i, j) \tag{2}$$

Image classification with CNNs — Convolution

- Image processing often utilizes the (discrete) convolution operation
- For a 2D image I and a kernel k the convolution operation $*$ is defined by

$$(I * k)(x, y) = \sum_{i,j=-n}^{n} I(x - i, y - j)k(i, j) \tag{2}$$

- The (mirrored) kernel k is shifted over the image I and weighted sums are computed and stored in a new image

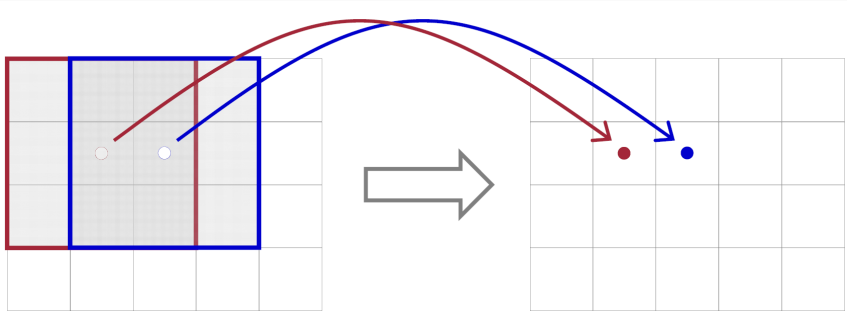


Image classification with CNNs — Convolution

Exercise

$$I = \begin{bmatrix} 2 & 4 & 6 & 8 \\ 1 & 3 & 5 & 7 \\ 0 & 2 & 4 & 6 \\ 0 & 1 & 2 & 6 \end{bmatrix} \quad k = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Task: Compute the convolution $I * k$

Resulting image size: 2×2

$$(I * k) = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix}$$

Image classification with CNNs — Convolution

- convolutions are commonly used in conventional image analysis, since they consider local neighborhoods in images

Image classification with CNNs — Convolution

- convolutions are commonly used in conventional image analysis, since they consider local neighborhoods in images
- they can be used for smoothing images

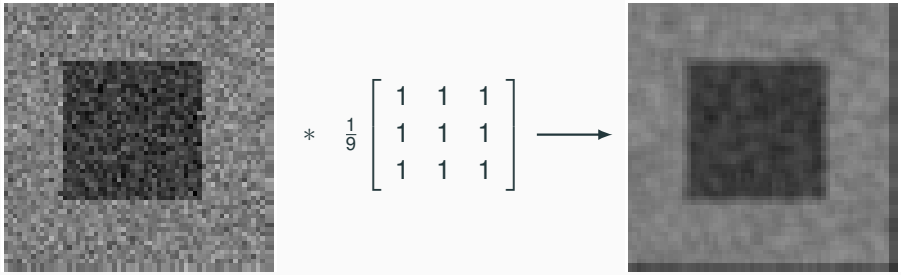


Image classification with CNNs — Convolution

- convolutions are commonly used in conventional image analysis, since they consider local neighborhoods in images
- they can be used for smoothing images
- each kernel can extract special features from the image

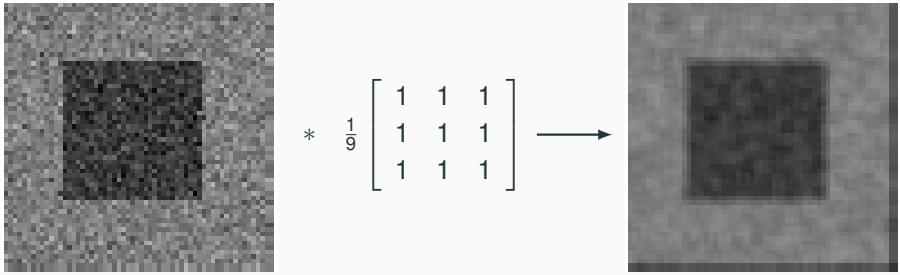
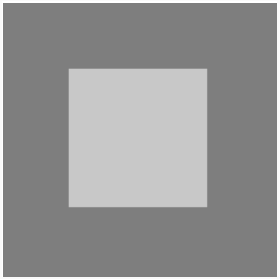


Image classification with CNNs — Convolution

- convolutions are commonly used in conventional image analysis, since they consider local neighborhoods in images
- they can be used for smoothing images
- each kernel can extract special features from the image



$$* \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \rightarrow$$

Image classification with CNNs — Convolution

- convolutions are commonly used in conventional image analysis, since they consider local neighborhoods in images
- they can be used for smoothing images
- each kernel can extract special features from the image, like horizontal edges

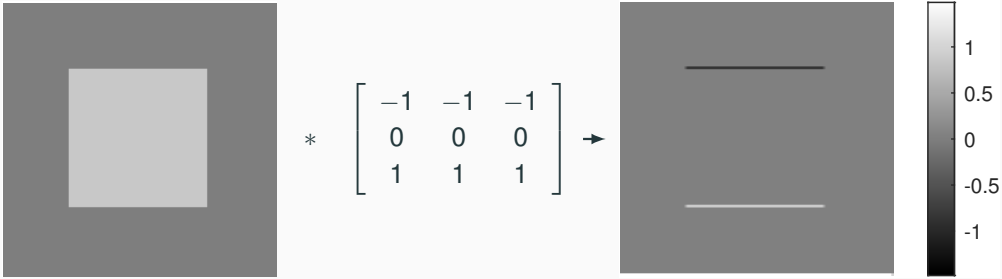


Image classification with CNNs — Convolution

- convolutions are commonly used in conventional image analysis, since they consider local neighborhoods in images
- they can be used for smoothing images
- each kernel can extract special features from the image, like horizontal edges or vertical edges

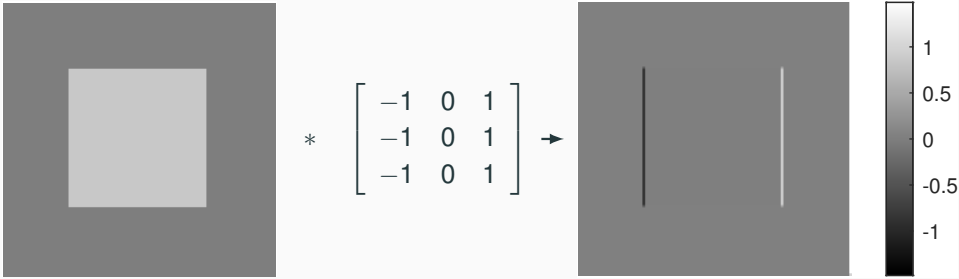


Image classification with CNNs — Neurons

- CNNs are regression models consisting of several “neurons”

Y. Bengio, I. Goodfellow, A. Courville. Deep Learning. MIT press, 2017.

Image classification with CNNs — Neurons

- CNNs are regression models consisting of several “neurons”
- Each neuron has consists of kernels $k_1, \dots, k_n$ which processes ingoing images $I_1, \dots, I_n$ via convolutions

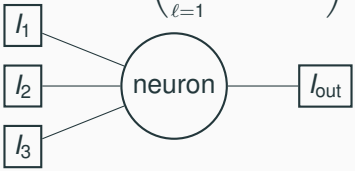
$$I_{\text{out}} = f\left(\sum_{\ell=1}^{n} I_\ell * k_\ell + b\right) \tag{3}$$

Y. Bengio, I. Goodfellow, A. Courville. Deep Learning. MIT press, 2017.

Image classification with CNNs — Neurons

- CNNs are regression models consisting of several “neurons”
- Each neuron has consists of kernels $k_1, \dots, k_n$ which processes ingoing images $I_1, \dots, I_n$ via convolutions

$$I_{\text{out}} = f\left(\sum_{\ell=1}^{n} I_\ell * k_\ell + b\right) \tag{3}$$

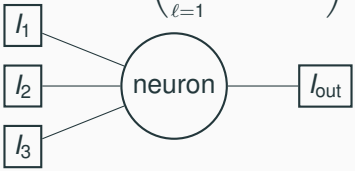


Y. Bengio, I. Goodfellow, A. Courville. Deep Learning. MIT press, 2017.

Image classification with CNNs — Neurons

- CNNs are regression models consisting of several “neurons”
- Each neuron has consists of kernels $k_1, \dots, k_n$ which processes ingoing images $I_1, \dots, I_n$ via convolutions

$$I_{\text{out}} = f\left(\sum_{\ell=1}^{n} I_\ell * k_\ell + b\right) \tag{3}$$



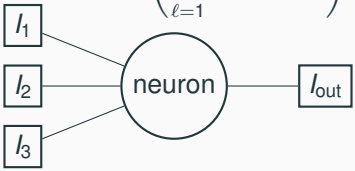
- The kernels $k_1, \dots, k_n$ and the scalar b are regression (trainable) parameters

Y. Bengio, I. Goodfellow, A. Courville. Deep Learning. MIT press, 2017.

Image classification with CNNs — Neurons

- CNNs are regression models consisting of several “neurons”
- Each neuron has consists of kernels $k_1, \dots, k_n$ which processes ingoing images $I_1, \dots, I_n$ via convolutions

$$I_{\text{out}} = f\left(\sum_{\ell=1}^{n} I_\ell * k_\ell + b\right) \tag{3}$$



- The kernels $k_1, \dots, k_n$ and the scalar b are regression (trainable) parameters
- f is called the activation function, e.g., $f = \tanh$ or $f = \mathrm{ReLu}$.

Y. Bengio, I. Goodfellow, A. Courville. Deep Learning. MIT press, 2017.

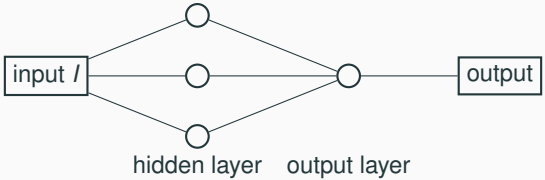
Image classification with CNNs — Convolutional layers

CNNs are an aggregation of such neurons

- The first layer is called the input layer, the last layer is called the output layer, and the layers in between are called hidden layers.

Image classification with CNNs — Convolutional layers

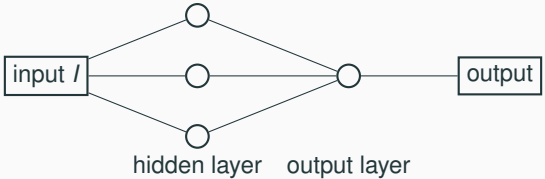
CNNs are an aggregation of such neurons



- The first layer is called the input layer, the last layer is called the output layer, and the layers in between are called hidden layers.
- In this minimalistic example, we have one hidden layer with three neurons and an output layer with one neuron.

Image classification with CNNs — Convolutional layers

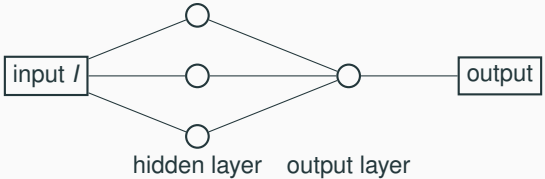
CNNs are an aggregation of such neurons



- The first layer is called the input layer, the last layer is called the output layer, and the layers in between are called hidden layers.
- In this minimalistic example, we have one hidden layer with three neurons and an output layer with one neuron.
- The input I is processed by the three neurons, each of which convolves the input with its own kernel, resulting in three images in the hidden layer

Image classification with CNNs — Convolutional layers

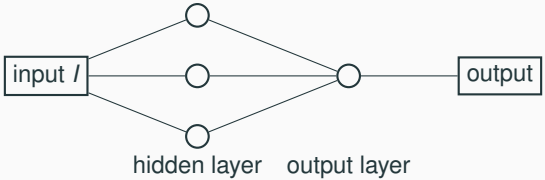
CNNs are an aggregation of such neurons



- The first layer is called the input layer, the last layer is called the output layer, and the layers in between are called hidden layers.
- In this minimalistic example, we have one hidden layer with three neurons and an output layer with one neuron.
- The input I is processed by the three neurons, each of which convolves the input with its own kernel, resulting in three images in the hidden layer
- Output layer convolves each of the three images with its own kernel

Image classification with CNNs — Convolutional layers

CNNs are an aggregation of such neurons

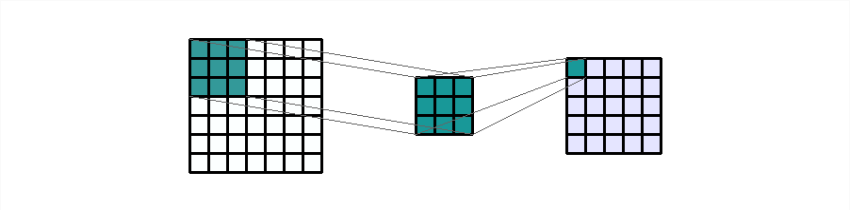


- The first layer is called the input layer, the last layer is called the output layer, and the layers in between are called hidden layers.
- In this minimalistic example, we have one hidden layer with three neurons and an output layer with one neuron.
- The input I is processed by the three neurons, each of which convolves the input with its own kernel, resulting in three images in the hidden layer
- Output layer convolves each of the three images with its own kernel
- Typically, the output of a hidden layer is considered to be a feature map, in this example, an image with three channels.

Image classification with CNNs — Convolutional layers

Torch API

```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,
padding=0, dilation=1)
```

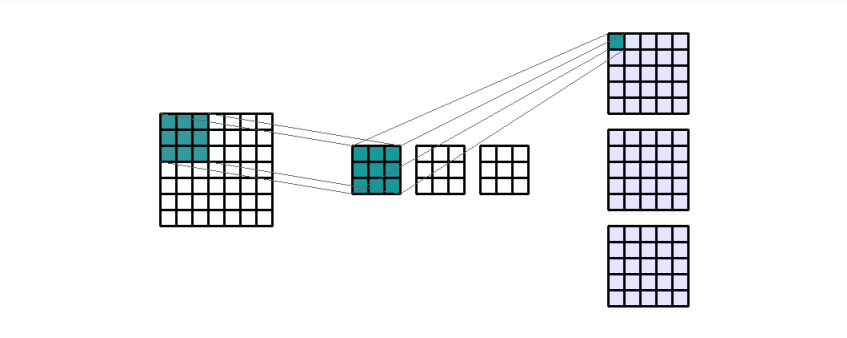


kernel_size: Defines the spatial size of the convolution filter (e.g. 3×3). Influences receptive fields of neurons.

Image classification with CNNs — Convolutional layers

Torch API

```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,
padding=0, dilation=1)
```

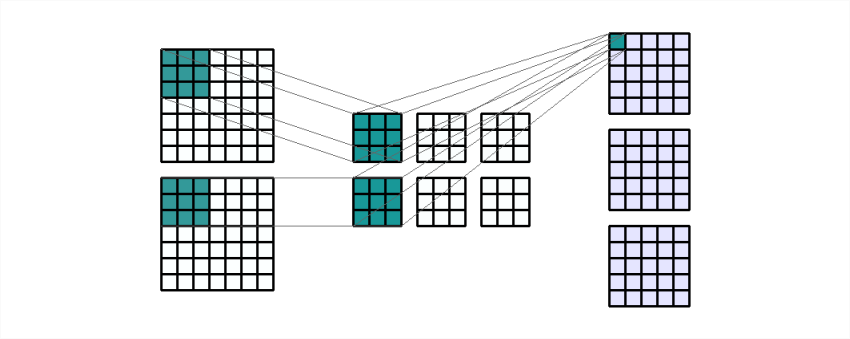


out_channels: Number of output feature maps (e.g., 3) produced by the layer (number of neurons)

Image classification with CNNs — Convolutional layers

Torch API

```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,
padding=0, dilation=1)
```

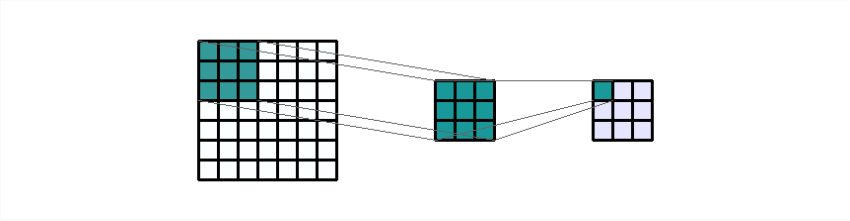


in_channels: Number of input feature maps (e.g., 2).

Image classification with CNNs — Convolutional layers

Torch API

```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,
padding=0, dilation=1)
```

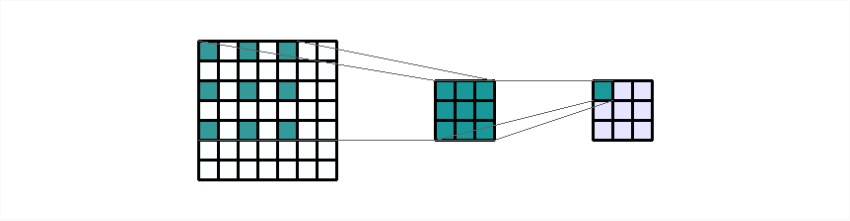


stride: Determines how far the kernel moves at each step (e.g., 2). Downsamples the output.

Image classification with CNNs — Convolutional layers

Torch API

```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,
padding=0, dilation=1)
```

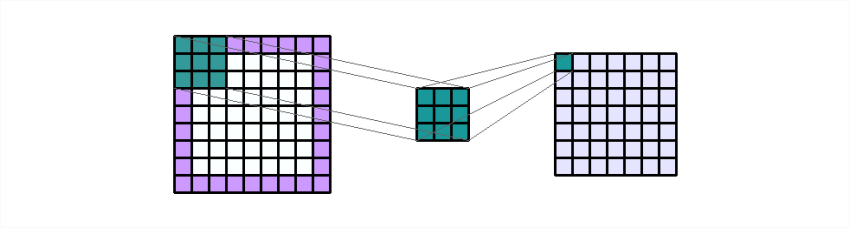


dilation: Introduces spacing between kernel elements (dilated convolution). Effectively increases the kernel size, without increasing the number of parameters.

Image classification with CNNs — Convolutional layers

Torch API

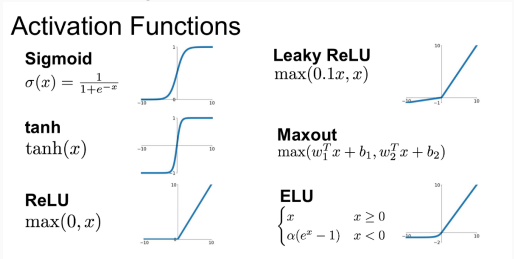
```
torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1,
padding=0, dilation=1)
```



padding_mode: Adds pixels (typically with values of 0) around the input boundary, e.g., such that output has the same spatial dimension as input.

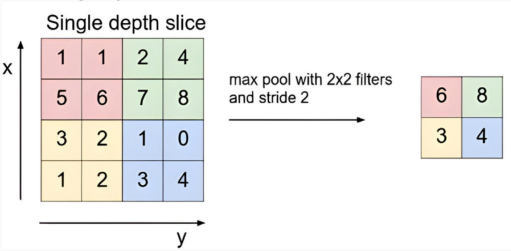
Image classification with CNNs — Activation and pooling layers

Activation layer



- typically applied element-wise to feature maps
- introduces **nonlinearity**
- allows CNNs to learn complex patterns

Pooling layer

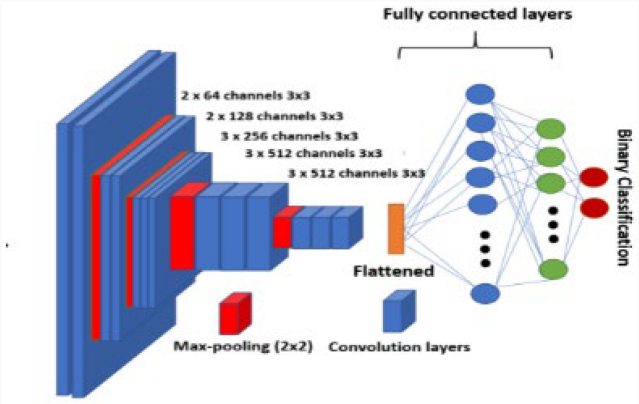


- reduces spatial resolution
- allows “small kernels” to capture larger receptive fields
- typical choice: **max pooling**

Typical CNN block:

Conv2D $\rightarrow$ Activation $\rightarrow$ Pooling

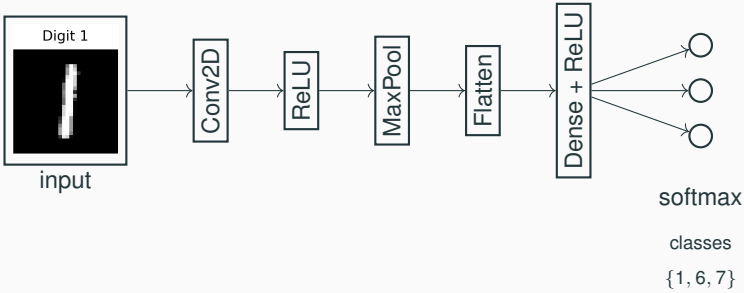
Image classification with CNNs — VGG networks



Idea of VGG networks

- Deep CNN using only small filters
- Repeated blocks of Conv $\rightarrow$ ReLU $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pooling
- Final layers perform classification

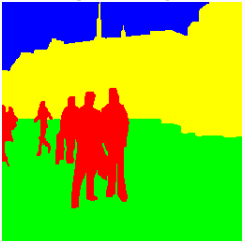
Image classification with CNNs — Minimalistic example



- Convolutional layers automatically learn features from the image.
- Pooling reduces spatial resolution while retaining important patterns.
- Dense layers perform the final classification.
- The softmax layer outputs probabilities for the three classes $\{1, 6, 7\}$.
- **Accuracy** for classifying 1, 6, 7 in the MNISt dataset: 0.96 (see jupyter notebook in itslearning)

Image segmentation — Data

Input image



Segmentation mask

- In image segmentation we assign a **class label to each pixel**.
- The training data consists of pairs of input image X and segmentation masks Y

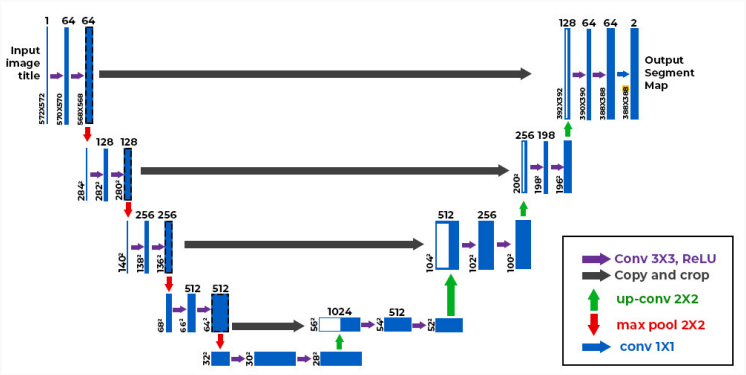
Pixel-wise labels

Y is one-hot-encoded, i.e., for each pixel (i, j) we have $Y(k, i, j) = 1$ if (i, j) is associated with class k and $Y(k, i, j) = 0$ otherwise.

Tensor conventions

Typically, the images X and Y are represented as tensors of shape (C, H, W) and (K, H, W) respectively, where H is the height, W is the width, and C is the number of channels (e.g., 3 for RGB images, or K for the output image).

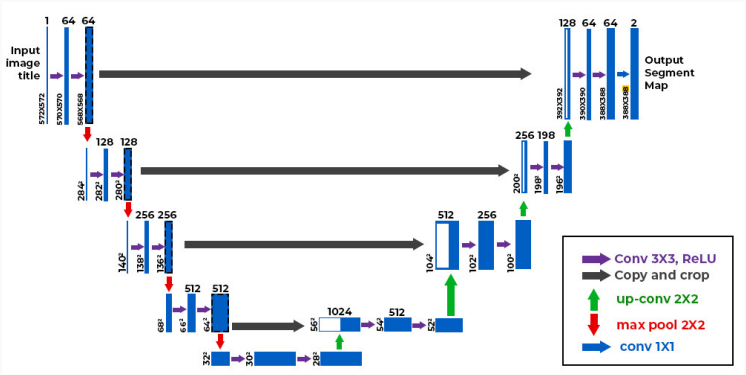
Image segmentation — U-Net architecture



U-Net architecture (encoder–decoder with skip connections). Numbers above the layers indicate the number of neurons. The number above the output layer is the number of classes K.

- U-Net is a convolutional neural network designed for **image segmentation**.

Image segmentation — U-Net architecture



U-Net architecture (encoder–decoder with skip connections). Numbers above the layers indicate the number of neurons. The number above the output layer is the number of classes K.

- U-Net is a convolutional neural network designed for **image segmentation**.
- The architecture has two main parts: Encoder and Decoder

Image segmentation — U-Net architecture

Encoder (contracting path)

- Consists of blocks of Conv2D $\rightarrow$ ReLU $\rightarrow$ Conv2D $\rightarrow$ ReLU
- Max pooling reduces spatial resolution $(C, H, W) \rightarrow (C, H/2, W/2)$
- After downsampling the number of feature channels is typically doubled
- This produces increasingly abstract feature representations.

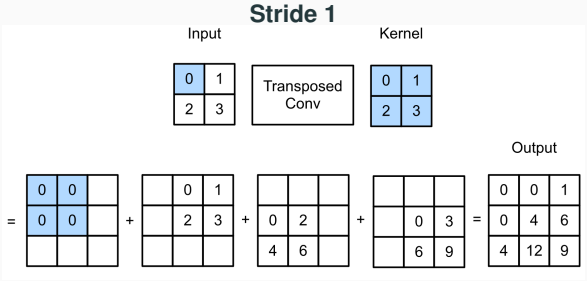
Decoder (expanding path)

- Upsampling (transpose convolution) is used to increase spatial resolution and decrease number of channels $(C, H, W) \rightarrow (C/2, 2H, 2W)$
- Skip connections concatenate encoder features with decoder features
- Convolutions combine abstract features with finer resolved features (from skip connections).

Output layer

- For each pixel (i, j) the network outputs scores $z(i, j, k)$ for each class k.
- A pixel-wise softmax can converts the scores into probabilities $\widehat{Y}(k, i, j) = \frac{e^{z(k,i,j)}}{\sum_{\ell=1}^{K} e^{z(\ell,i,j)}}$ that can be used for training
- After training segmentation mask S is obtained via $\hat{S}(i, j) = \arg\max_k \widehat{Y}(k, i, j)$

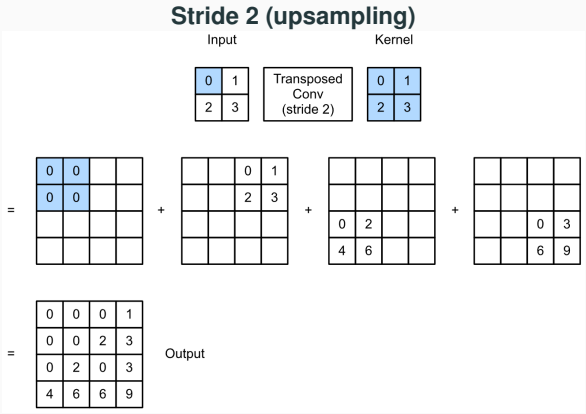
Image segmentation — Transposed convolution



Transposed convolution

- Used in the decoder of U-Net to **increase spatial resolution**.
- The operation distributes the kernel contributions over a larger output grid.

Image segmentation — Transposed convolution



Transposed convolution

- Used in the decoder of U-Net to **increase spatial resolution**.
- The operation distributes the kernel contributions over a larger output grid.
- With stride 2, the spatial dimensions increase

$$(H, W) \to (2H, 2W)$$

- The kernel weights are learned during training.
- This operation is sometimes called **deconvolution** or **fractionally strided convolution**.

Image segmentation — Training

Pixel-wise loss

- The network predicts class probabilities $\widehat{Y}(k, i, j)$ for pixel (i, j) and class k.
- The ground truth mask is $Y(k, i, j) \in \{0, 1\}$
- Training is performed using the **pixel-wise cross-entropy loss** $L = -\sum_{i,j} \sum_{k=1}^{K} Y(i, j, k) \log \widehat{Y}(i, j, k)$.

Mini-batch training

- Instead of computing the loss over the entire dataset, training is performed on **mini-batches**.
- Each batch contains several images and their corresponding masks.
- The loss is averaged over all pixels in the batch.

Data augmentation

To improve generalization, additional training samples are generated by applying transformations such as

- rotations and flips
- cropping or scaling
- intensity variations

to both the image and the corresponding segmentation mask.

Evaluation

Classification metrics from **Module 2** can be applied **for each pixel** followed by averaging.